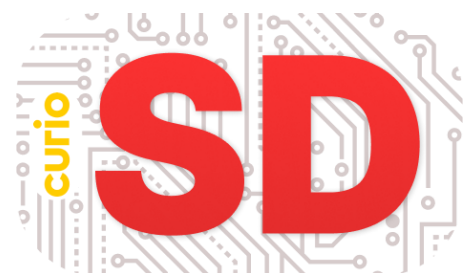


Reader WED-II

Werken als developer



Over dit document

Opleiding: Software Developer
Blok: C
Vak(ken): PRO

Versiebeheer

Versie	Datum	Auteur	Aanpassingen
1.0	17-02-2022	Kitty Martens	Opzet module
2.0	20-07-2022	Kitty Martens	Flowchart ipv Activity Diagram
3.0	07-03-2023	Bart Roos Eva van Os	Studiewijzer vernieuwd, content herschreven (tussenoplossing voor 23-feb)
4.0	10-07-2023	Bart Roos	Geheel nieuwe indeling PRO blok C, nieuwe studiewijzer

Inhoudsopgave

I. Over deze module	2
II. Studiewijzer	3
III. Studiepunten.....	4
1. Projectmatig en 'agile' werken	5
2. Scrum.....	6
3. Interview voorbereiden en uitvoeren	8
4. Product backlog en user story's	11
5. Sprintbacklog of scrumbord	14
6. Wireframes	16
7. Technisch testen.....	18

I. Over deze module

Doelstelling en relevantie

Deze module behandelt:

- De basis van scrum (*roles, rituals, artifacts*),
- Het proces van plannen/ontwerpen volgens het V-model, met speciale aandacht voor:
 - Het doen van onderzoek (dit als voorbereiding op C3),
 - Opstellen van user stories,
- Technisch testen (in verhouding met andere soorten testen).

Werkwijze

Dit boekje is slechts een verzameling van bronnen en opdrachten. Werk deze door op aanwijzing van de docent. De bronnen zijn gesorteerd op thema, niet op de volgorde van de lessen. Je kunt het boekje natuurlijk wel zelfstandig gebruiken als naslagwerk, bijvoorbeeld als je voor een toets wil leren.

II. Studiewijzer

Opdrachten met het -icoon betreffen een feedbackmoment, je ontvangt hiervoor studiepunten.

Wk	Onderwerpen / lesdoelen	Bronnen / opdrachten
1	A) Lessenreeks opstarten / ontwaken uit de vakantie. B) Begrijpen wat projectmatig werken is (versus: routine, improvisatie). C) De soorten projectmatig werken kennen (waterval en agile). D) Begrijpen dat scrum slechts één implementatie van agile is. E) Belangrijkste kenmerken van scrum weten (roles, rituals, artifacts). F) Inzien dat je een aantal delen van het scrumproces al kent.	H1, H2
2	A) De rollen, ceremonies en producten van scrum kennen. B) Begrip van de gesprekstechniek Luisteren, Samenvatten, Doorvragen. C) Duidelijk beeld hebben van waar je naar vraagt in het interview. D) Weten hoe je een lijst van doelgroepen uitwerkt, evt. als persona's. E) Introductie van de eerste casus.	H2, H3
3	<i>Gelijk aan vorige les.</i>	H2, H3 Uitvoeren interview
4	A) De rollen, ceremonies en producten van scrum kennen. B) Weten wat een product backlog (item) is. C) Weten dat een PBI als User Story wordt geschreven. D) Kennen format user story. E) Begrijpen dat iedere persona een of meer story's krijgt.	H4 User story's inleveren
5	<i>Gelijk aan vorige les, en daarnaast:</i> A) Een complete en correcte lijst story's kunnen opstellen. B) Weten wat wireframes zijn en de criteria voor opstellen kennen	H6  FESD (1 ^e casus)
6	A) Timeboxes voor het scrumproces kennen. B) Weten wat acceptatiecriteria zijn. C) Herhaling scrum, proces echt gaan invoelen.	Bron 2.4 Bron 4.3
7	A) Je begrijpt dat een Productbacklog-item (PBI) nog geen taak is. B) Je weet hoe je van een PBI taken kunt maken. C) Je kent de timebox van een taak en kunt deze verklaren. D) Je kent de indeling van een scrumbord. E) Je weet wat een definitief-of-done is. F) Je kunt onderscheiden wat er wel / niet in een DoD hoort.	Bron 2.4 Bron 4.4 H5
8	A) Je begrijpt het nut van vooronderzoek.	Bron 3.4
9	-	 FYAK (scrumtoets)
10	-	 FSVE (2 ^e casus)
11	A) Je begrijpt het concept van een technische test. B) Je kunt de happy path/edge cases/extreme cases aanduiden. C) Je kunt een technische test opstellen.	H7
12	<i>Gelijk aan vorige les.</i>	 FH3W (tech. testen)

III. Studiepunten

A-punten

Zie het studiepuntenplan op *Itslearning* voor de precieze checks.

Code	Week	Onderwerp	Punten	Cesuur
FESD	5	Eerste casus	2	80%
FYAK	9	Scrumtoets	2	70%
FSVE	10	Tweede casus	2	70%
FH3W	12	Theorietoets technisch testen	1	70%

B-punten

Enkele aandachtspunten voor het behalen van de twee B-punten voor dit vak:

- Actief meedoen in de lessen over het algemeen,
- Proactief zijn bij problemen, uitdagingen of vragen rondom de feedbackmomenten.

1. Projectmatig en 'agile' werken

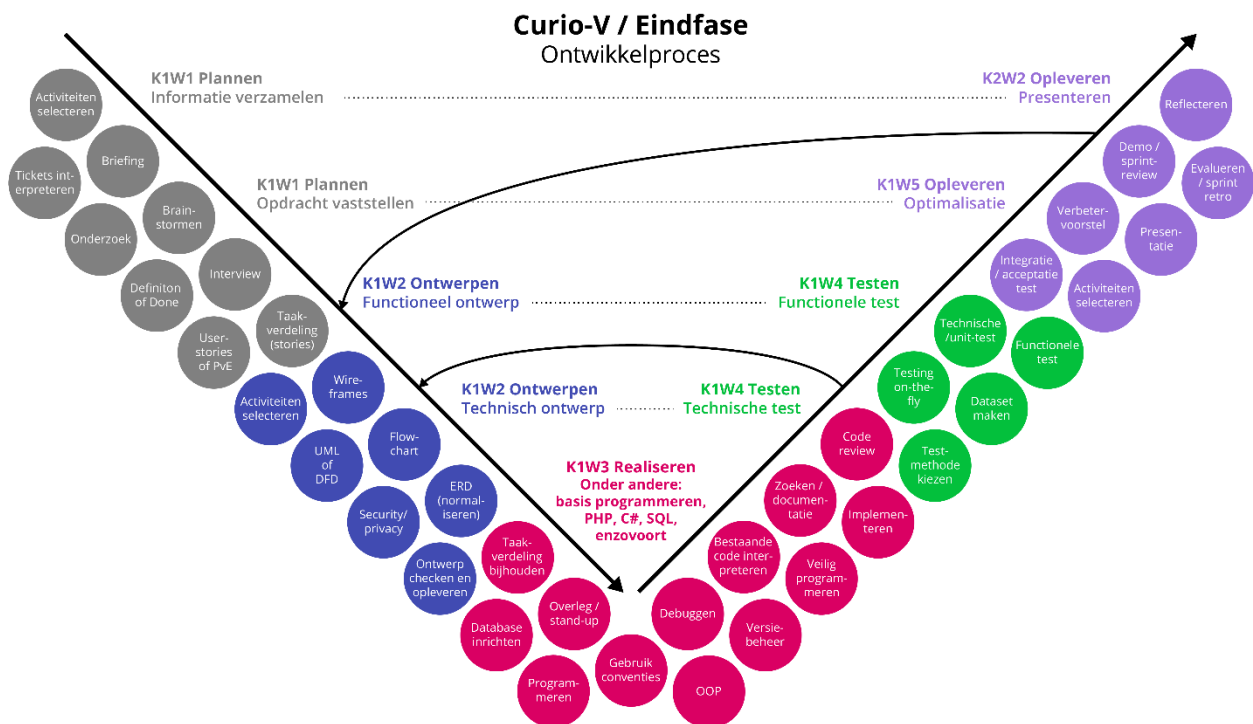
Er zijn grofweg drie manieren waarop je kunt werken:

1. Routinematig: bijvoorbeeld vakkenvullen in de supermarkt; éénmaal een trucje leren en dat steeds blijven toepassen, je krijgt routine in het werk en er komen niet veel verrassingen.
2. Improvisatiematig: bijvoorbeeld het werk als docent; op basis van je gaat doen veel kennis en degelijke voorbereiding neem je steeds ter plekke besluiten over wat je precies gaat doen.
3. Projectmatig: meer vooraf gepland, met een team naar een vast doel toewerken. Een project heeft een bepaalde tijdsduur, het is na een paar weken of maanden meestal afgelopen (omdat de app klaar is, de klant tevreden is of simpelweg omdat het geld op is). Daarna start eventueel een nieuw project van voor af aan.

Binnen de Software Development wordt eigenlijk altijd projectmatig gewerkt. Bij projectmatig werken horen een aantal vaste stappen. In de software development zijn die stappen uitgewerkt in het V-model.

? Bron 1.1. Het V-model

Aan de binnenkant staan de hoofdfases vermeld (plannen, ontwerpen, realiseren, testen, opleveren). Aan de buitenkant staan verschillende activiteiten die je per fase zou kunnen ondernemen.



De naam "model" geeft aan dat het een versimpelde weergave van de echte wereld is. Het komt niet vaak voor dat je exact op deze volgorde precies deze activiteiten uitvoert in een project. Maar het is wel een handig "praatplaatje" om de globale stappen van een project te leren begrijpen.

Binnen het projectmatig werken zijn er grofweg twee aanpakken:

1. Waterval: aan het begin van een project maak je een planning, ontwerpen, enzovoort. Daarna ga je voor een langere periode ontwikkelen terwijl je geen contact hebt met de klant. Aan het einde van het project is er een oplevering. De nadelen zijn duidelijk; weinig contact met de klant en dus weinig bevestiging dat je op de goede weg zit.
2. Agile: na een korte opstartfase bouw je een stuk van het project in een sprint van meestal 2 tot 4 weken. Dat lever je op aan de klant, alvorens met het volgende stuk verder te gaan. Er is veel ruimte om bij te sturen, je bent 'wendbaar' (letterlijke vertaling van agile).

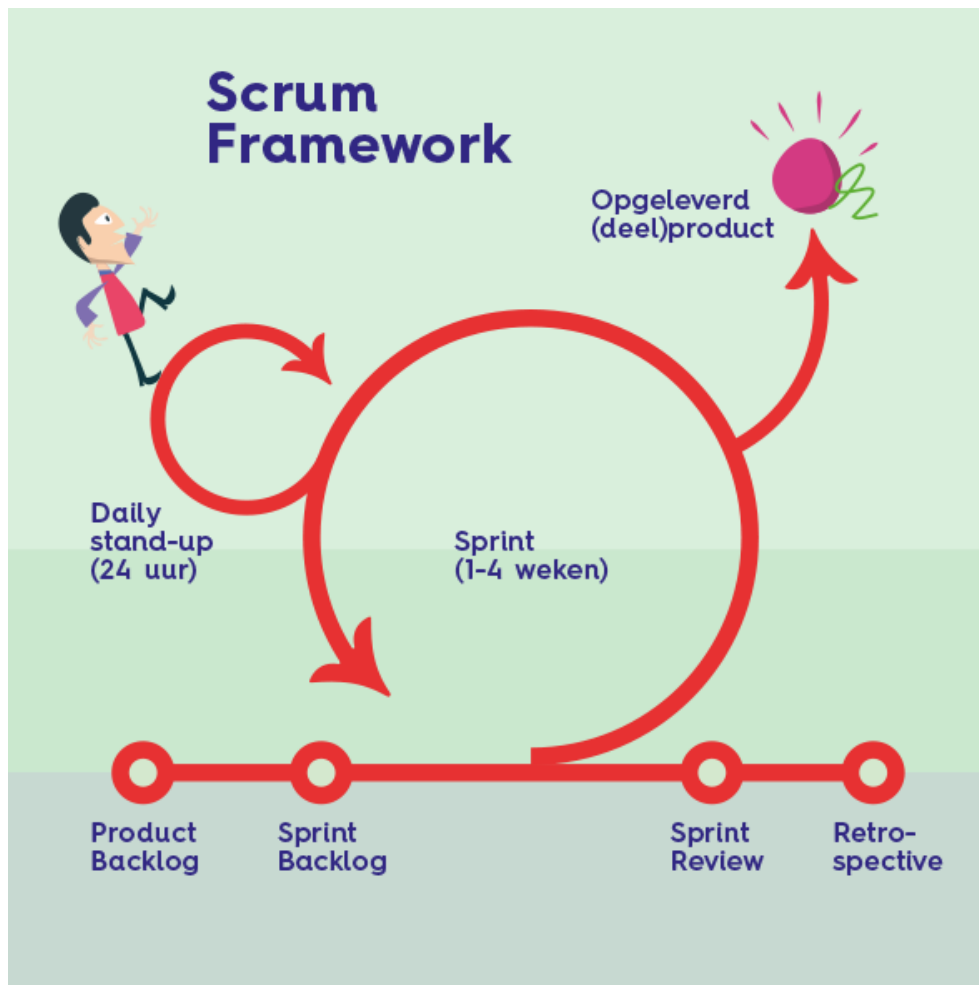
2. Scrum

Agile is een denkwijze, een nog redelijk vage aanpak die alleen zegt dat je in korte sprints moet werken en dat je open moet staan voor bijsturen van het project, en dat je veel contact met de klant moet hebben.

Scrum is een meer specifieke implementatie van agile die veel concreter vastlegt dat een sprint bijvoorbeeld maximaal 4 weken mag duren, en ook welke overleggen er zijn, welke producten je oplevert, enzovoort.

Scrum is dus een set afspraken over hoe je een project op een agile manier kunt aanpakken.

? Bron 2.1. Scrum in een notendop



Roles	Artifacts	Ceremonies
• Product owner • Development team • Scrum master	• Increment • Product backlog • Sprint backlog	• Sprint planning • Sprint review • Sprint retrospective • Daily scrum

? **Bron 2.2. Scrum in 7 minuten**
<https://www.youtube.com/watch?v=9TycLR0TqFA>

? **Bron 2.3. The Official Scrum Guide**
De “uitvinders” van Scrum (Jeff Sutherland en Ken Schwaber) publiceren hun officiële visie op scrum gratis online. Dit is een verrassend goed leesbare en compacte gids. Vooral geschikt als naslagwerk, wanneer je bijvoorbeeld de exacte bedoeling van een “sprint retrospective” nog eens wil nalezen.

- Officiële versie (Engels): <https://scrumguides.org/scrum-guide.html>
- Vertaalde versie (Nederlands, pdf): <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Dutch.pdf>

? **Bron 2.4. Timeboxes**
Alle gebeurtenissen binnen een scrumproject hebben een ‘timebox’, ofwel een maximale tijd waarbinnen de bijeenkomst moet plaatsvinden. Scrum is hierin erg strikt; als de tijd om is dan is de meeting afgelopen en moet je aan de slag. Dit om te voorkomen dat je alsnog (zoals bij waterval) veel tijd verliest aan praten en plannen maken, in plaats van aan een concreet product bouwen.

Naast de officiële timebox, geven we hieronder ook aan wat we binnen de opleiding hanteren. De officiële timeboxes gaan immers uit van een werkweek van 40 uur. In de opleiding werk je maar enkele uren per week aan een praktijkopdracht, en dus zijn de projecten kleiner en de timeboxes soms korter.

	Timebox (officieel)	Timebox (opleiding)
Sprint	1 tot 4 weken	1 tot 4 weken
Sprint planning	2 uur per week van de sprint	20 minuten per week van de sprint
Eén taak op bord	1 werkdag	1 praktijkles
Daily stand-up	15 minuten	15 minuten
Sprint review	2 tot 4 uur	10 tot 20 minuten
Sprint retrospective	60 tot 90 minuten	10 tot 20 minuten

3. Interview voorbereiden en uitvoeren

Een goed interview is eigenlijk een gesprek. Het is niet het afvuren van vragen zodat je dat maar gehad hebt. Natuurlijk stel je vragen op, maar deze zijn een richtlijn voor het gesprek dat je hebt. Je moet ook goed luisteren naar de antwoorden en eventueel doorvragen. Dat alles gaat eens stuk beter als je open vragen stelt. Deze aanpak noemen wel ook wel een gesprekstechniek, afgekort als “LSD”.

? **Bron 3.1. Luisteren, Samenvatten, Doorvragen**

De belangrijkste regel voor een goed interview is het toepassen van “LSD”, dat begint met het stellen van een open vraag en daarna:

- **Luisteren** – luister aandachtig naar het antwoord
- **Samenvatten** – vat het antwoord samen, check of je het goed begrepen hebt
- **Doorvragen** – stel eventueel vervolgvragen

Voorbeeld van het toepassen van LSD:

Interviewer (open vraag): “Welke gegevens moeten we opslaan voor een collega die een weddenschap wil plaatsen?”

Opdrachtgever: “Ik denk dat de naam voldoende is in dit geval, omdat het toch collega’s onder elkaar is. We hoeven verder geen details te weten eigenlijk. Hoewel, het zou wel handig kunnen zijn om ook naar het e-mailadres te vragen. Dat is misschien wel nodig voor een inlogaccount of zoiets.” (interviewer luistert aandachtig)

Interviewer (samenvatten): “Als ik het goed begrijp moet de applicatie dus naam en e-mailadres vragen?”

Klant: “Ja, dat lijkt me wel correct.”

Interviewer (doorvragen): “U noemde ook een inlogaccount, is dat een functie die nu al ingebouwd moet worden?”

Klant: “Nee, nu nog niet. Maar ontwerp de applicatie zo dat het later eventueel wel kan.”

? **Bron 3.2. Interviewverslag**

Tijdens het interview schrijf je mee met de antwoorden (als je met een groep bent, verdeel dan de taken zodat één iemand het gesprek leidt en de ander schrijft). Maar een goed interviewverslag is meer dan de letterlijke antwoorden op de vragen.

De inhoudsopgave van het verslag van je interview zou er als volgt uit kunnen zien:

1. **Inleiding:** datum, locatie, aanwezigen.
2. **Beschrijving van het bedrijf:** wat is het voor soort bedrijf, welke doelstelling heeft het bedrijf, grootte van het bedrijf, enzovoort.
3. **Huidige situatie:** wat is het probleem, welke processen gaan er nu niet goed binnen het bedrijf, welke kansen worden er niet benut (nieuwe ontwikkelingen).
4. **Gewenste situatie:** hoe is het proces in de ideale situatie, als het probleem is opgelost, hoe werkt het bedrijf dan, hoe gebruiken ze de app die je gaat bouwen om het probleem op te lossen.
5. **Doelgroepen:** wie gaan er met de app werken? Je moet minstens de verschillende groepen gebruikers aanduiden die samen dezelfde eisen hebben (bijvoorbeeld: gebruikers en beheerders, of leerlingen en docenten, of kassamedewerkers en managers, enzovoort).

Als je deze dingen in je verslag wil zetten, moet je dus zorgen dat je hiernaar hebt gevraagd.

? Bron 3.3. Persona's als uitwerking van de doelgroep

Naast een lijst met doelgroepen, kan het soms goed zijn om je mogelijke gebruikers nog verder uit te werken. De bedoeling is, dat door de beschrijving van de doelgroep, het team zich beter kan identificeren met de gebruikers. Zo zal het team beter begrijpen hoe de gebruikers denken en wat ze van de applicatie verwachten.

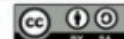
Binnen scrum wordt daarvoor de Persona-template gebruikt.



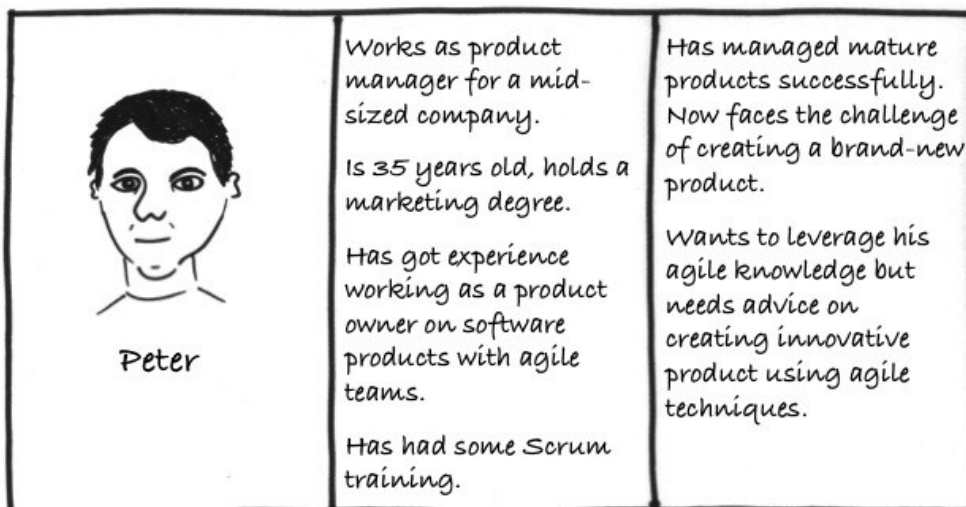
De persona's zijn een beschrijving van de gebruikers van de applicatie. Het is een beschrijving van één gebruiker of een groep gebruikers met de dezelfde functie.

📷 PICTURE & NAME	🔍 DETAILS	🎯 GOAL
What does the persona look like? What is its name? Choose a picture and a name that are representative, and that allow you to develop sympathy for the persona.	What are the persona's relevant characteristics and behaviours? Consider demographics, job, lifestyle, spare time activities, attitudes, and common tasks, for instance.	Why would the persona want to buy or use the product? What problems should the product solve? What benefits does the persona want to achieve? If there are multiple problems or benefits, identify the main one and put it at the top.

<http://www.romanpichler.com/>



Voorbeeld Persona:



? **Bron 3.4. Onderzoek doen als voorbereiding**

Als developer kun je in veel verschillende sectoren komen te werken, overal heeft men immers programmeurs en applicaties nodig. Denk aan de gezondheidszorg, de financiële wereld, in een fabriek of bij een webwinkel. De mogelijkheden zijn eindeloos.

Laten we inzoomen op één voorbeeld; stel je voor dat je binnenkort stage gaat lopen bij een hypotheekadviseur. Ze hebben daar een applicatie waarin ze allerlei berekeningen doen voor hun klanten, op basis van inkomen, aankoopssom en de hypotheekrente. Jij moet onderhoud gaan verrichten aan de applicatie; allerlei kleine uitbreidingen bouwen en wat bugs oplossen.

Volg je het nog? De kans dat je als student weet hoe een hypotheek in elkaar steekt is erg klein. Toch kun je zonder een beetje kennis daarover niet fatsoenlijk aan de slag op deze stage. Als je een bug moet oplossen in één van de berekeningen, dan moet je natuurlijk wel een idee hebben van waar die berekening over gaat.

Zo zijn er nog tientallen voorbeeld van stagebedrijven die heel interessant werk doen, maar waar je als gewone student eigenlijk niets van de inhoud van dat werk weet. Stel je nu eens voor dat je een opdracht doet voor zo'n bedrijf, en zonder enige voorbereiding een interview gaat doen met je klant. Dat kan haast niet goed gaan. Als je niets weet van het onderwerp dan weet je ook niet wat je moet vragen, en dan krijg je een erg ongemakkelijk interview.

Het is dus gebruikelijk om aan het begin van een opdracht even jezelf in te lezen in het bedrijf en de sector waarin ze werken. Tot nu toe heb je in de opleiding alleen maar opdrachten gedaan waar je als gewone student waarschijnlijk al iets over wist. Denk maar terug; we hebben onder andere gewerkt aan een pretparkwebsite, een webshop, een kassasysteem, een takenlijst, enzovoort.

Op technisch vlak waren die opdrachten misschien wel uitdagend, maar je begreep waarschijnlijk prima waar het over ging. Iedereen heeft wel eens takenlijstje gemaakt op papier, en als developer werk je ook vaak met projectborden die hetzelfde principe volgen als die opdracht.

Maar in de rest van je opleiding en in de stages ga je steeds vaker opdrachten krijgen waarbij je eigenlijk niets weet over de *inhoud*, dus het proces van de klant of het bedrijf waarvoor je werkt. Daarom moet je soms wat onderzoek doen, zodat je ongeveer weet waar een applicatie over moet gaan.

De leeruitkomst waar we naartoe werken is:

Je doet onderzoek ter voorbereiding van de opdracht of het interview; je verdiept je in het domein van de opdracht.

4. Product backlog en user story's

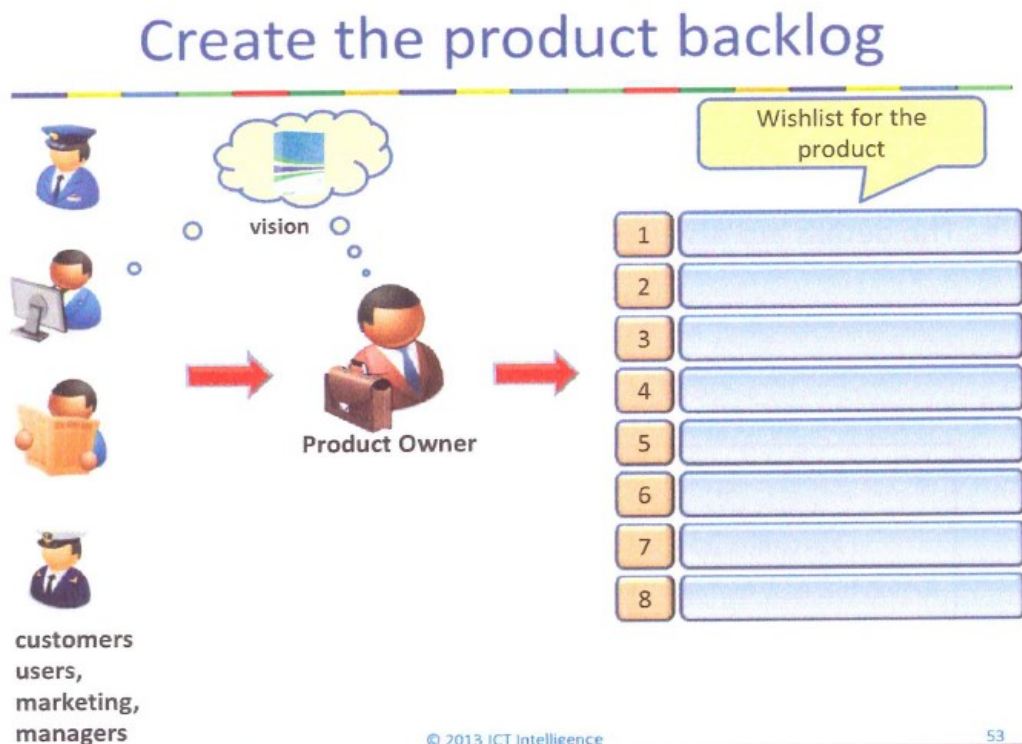
? **Bron 4.1. Productbacklog**

De Product Owner verzamelt eisen en wensen voor de applicatie bij de opdrachtgever maar eventueel ook bij eindgebruikers of andere 'stakeholders' (=belanghebbenden).

Al deze eisen en wensen komen om de Productbacklog. Een "backlog" is niks anders dan een lijst met wat er ooit moet gebeuren.

Een productbacklog is nog géén takenlijst, dat komt later. Een productbacklog kan namelijk ook vage ideeën bevatten die eerst verder moeten worden uitgewerkt.

Meestal hebben productbacklogitems (kortweg: PBI) de vorm van een user story (bijvoorbeeld: *als klant wil ik een item in mijn winkelmandje doen, zodat ik het kan bestellen*).



? **Bron 4.2. Wat is een user story?**

Scrum draait om het opleveren van werkende onderdelen, zodat je direct waarde kunt toevoegen. Bij scrum is een tevreden opdrachtgever de hoogste prioriteit. Wat ons eindproduct moet kunnen en doen, beschrijven we dus ook vanuit de *waarde* voor de *opdrachtgever*.

Een veel gebruikte manier van opschrijven is de “user story”, letterlijk te vertalen als ‘gebruikersverhaal’. Een user story is ook een klein verhaaltje over wat een eindgebruiker met de applicatie kan of wil doen.

Bijvoorbeeld: “als beheerder wil ik de uitslagen op kunnen halen, zodat ik de weddenschappen kan uitbetalen”.

Een user story gaat over wat en waarom, maar nog niet over het hoe (de technische oplossing).

Opstellen

Een user story gaat over de drie W's:

- **Wie** Als (... klant / gebruiker / enzovoort ...),
- **Wat** wil ik / kan ik / moet ik (... doel ...),
- **Waarom** zodat (... waarde van de story ...).

Je kijkt dus vanuit de eindgebruiker naar het product. Wat gaat de eindgebruiker willen of kunnen met de applicatie, en wat heeft hij daar vervolgens aan?

De user stories stel je op vanuit gesprek met de opdrachtgever, het interview dus. Het kan ook nuttig zijn om met eindgebruikers te spreken, als de opdrachtgever dat zelf niet is. Je moet namelijk een beeld krijgen van de dingen die *gebruikers* willen doen met de applicatie, en waarom dat belangrijk is (de ‘waarde’).

Het opstellen van de user stories doe je als team, het liefst samen met de Product Owner. In een gesprek komen de user stories tot stand, zodat er vanuit verschillende kanten input komt.



Voorbeelden

Als ...	Wil ik ...	Om te
Bezoeker festival	Webshop	Kaartjes te kunnen kopen
Bezoeker festival	Contactpagina	Om vragen te kunnen stellen
Bezoeker festival	Locatie	Om te kunnen zien waar het festival is
Bezoeker festival	Nieuws lezen	Om te blijven
Bezoeker festival	Artiesten (line up) / programmering	Om in te kunnen beslissen of ik naar het festival wil
Organisatie festival	Kaartjes verkopen	Festival te kunnen betalen
Organisatie festival	Weten hoeveel kaarten er zijn verkocht	Om het festival goed te kunnen organiseren

? **Bron 4.3. Acceptatiecriteria**

User story's zijn beschreven vanuit het perspectief van de gebruiker, en gaan nog niet over het ‘hoe’. Dat maakt wel dat ze soms wat vaag zijn. Het kan voorkomen dat een opdrachtgever toch bepaalde kaders wil stellen aan het hoe, bijvoorbeeld bij deze user story:

“Als beheerder wil ik de uitslagen op kunnen halen, zodat ik de weddenschappen kan uitbetalen”.

- De uitslagen moeten via een API worden opgehaald
- De API moet gratis beschikbaar zijn

De opdrachtgever geeft hierbij als eis dat de uitslagen van een API moeten komen, en dat je dus niet zelf uitslagen hoeft in te vullen na een race. Dit is een voorbeeld van een ‘acceptatiecriterium’. Daarbij geeft de opdrachtgever toch bepaalde kaders voor het ‘hoe’, bovenop het ‘wat’ en ‘waarom’ dat al in de story zelf zit.

Een user story is pas af wanneer de PO tevreden is én aan alle acceptatiecriteria is voldaan.

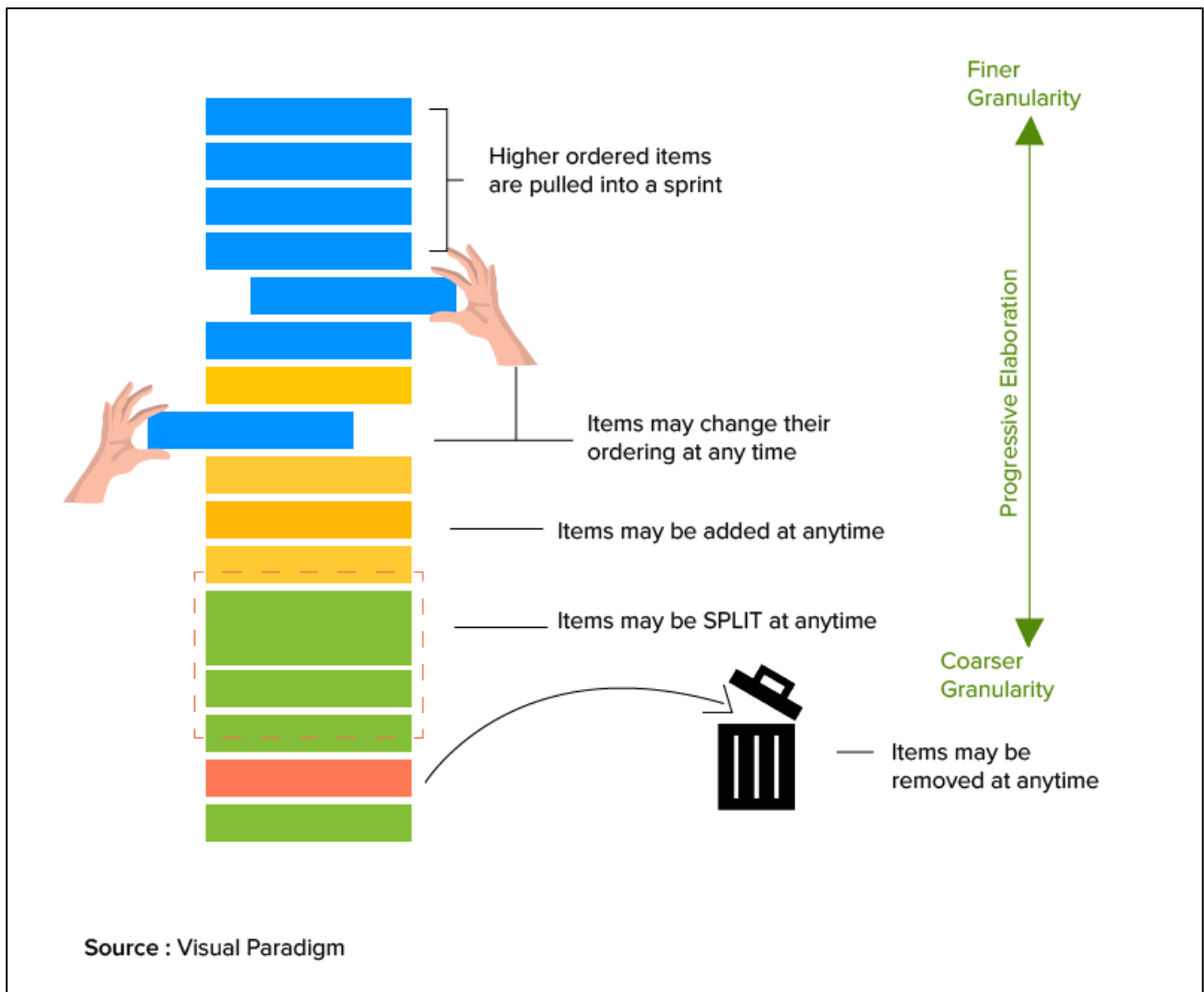
? **Bron 4.4. Van wie is de productbacklog?**

De productbacklog is een lijst die eigendom is van de Product Owner. Als developer houd je je hier niet zoveel mee bezig. De productbacklog bevat alle eisen, wensen, ideeën en 'dromen' over wat de app ooit zou moeten kunnen doen. De productbacklog bevat items die uiteenlopen tussen:

- Concreet, belangrijk en in de vorm van een user story, bijvoorbeeld: *“als gebruiker wil ik kunnen inloggen met mijn Google-account, zodat ik snel en eenvoudig met de applicatie aan de slag kan”*.
- Vage ideeën die ooit nog uitgewerkt kunnen worden, bijvoorbeeld: *“misschien kunnen we ooit iets doen met een api voor een bepaalde groep klanten o.i.d.”*

De PO zorgt ervoor dat deze lijst is gesorteerd op detail en prioriteit, dus: de meeste belangrijke en meest concrete backlog-items staan bovenaan. De vaagste en/of minst belangrijke items staan onderaan.

Het werk van de PO is feitelijk om ook de items onderaan uiteindelijk te verduidelijken en netjes in user story-vorm te zetten (door in gesprek te gaan met de klant, met eindgebruikers, enzovoort).



5. Sprintbacklog of scrumbord

? **Bron 5.1. Van productbacklog naar sprint**

Wanneer het development-team start aan een sprint, bepalen zij zelf (maar vaak wel in overleg met de PO) welke items ze van de productbacklog pakken. Het team bepaalt dit zelf, omdat ze alleen zelf weten hoeveel werk ze kunnen gaan verzetten in een sprint. Er is wel overleg met de Product Owner, omdat die natuurlijk weet welke items de hoogste prioriteit hebben, wat de klant nu het belangrijkste vindt, enzovoort.

? **Bron 5.2. Van story naar taken**

We weten dat de timebox van een taak één dag is (of binnen de opleiding: één praktijkles). Door die timebox zie je iedere dag beweging op je scrumbord. Of als het bord al een paar dagen stilstaat, dan zie je ook vrij snel dat er een probleem is of dat het tempo (veel) te laag ligt.

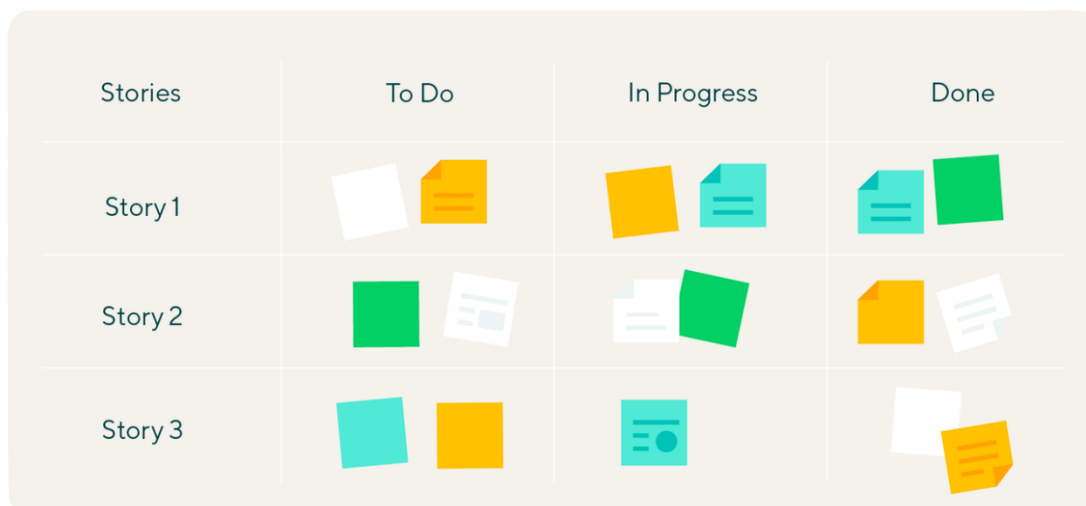
Het ontwikkelen van een hele user story (front-end, backend, enzovoort) kost wel vaak meer tijd dan die timebox van een dag / les. Daarom moet je een PBI opknippen in kleinere taken.

Een makkelijke manier om dat te doen is door te denken aan 'frontend/backend/database'. Bijvoorbeeld; ik moet een formulier maken, ik moet validatiecode schrijven, ik moet de tabel maken om in op te slaan.

? **Bron 5.3. Het scrumbord**

Het scrumbord delen we voortaan op deze manier in:

- Geheel links een kolom met alle PBI's / user story's die voor deze sprint zijn uitgekozen.
- In de kolom To Do plaats je taken die maximaal één dag / les groot zijn.
 - Tip: zorg dat je weet bij welke story de taak hoort, dat kan bijvoorbeeld met nummers, kleuren of door net als hieronder de taken op dezelfde hoogte als de story te plaatsen.
- Daarna komen nog de bekende kolommen:
 - In Progress
 - (Eventueel: In Review)
 - Done





Bron 5.4. Definition of done

Scrum draait om het toevoegen van *waarde* aan een product. Dat doe je door userstories af te maken, en zo steeds een beetje functionaliteit toe te voegen. Maar wanneer is een user story eigenlijk 'af'? Het is belangrijk dat iedereen in het team die vraag hetzelfde beantwoord. Als je het niet eens bent over wanneer een story 'klaar' is, dan kan daar conflict over ontstaan.

Acceptatiecriteria

We weten dat een user story 'acceptatiecriteria' kan hebben. Die zijn altijd specifiek voor die ene story. Bijvoorbeeld, de volgende acceptatiecriteria kunnen niet bij andere stories geplaatst worden:

"Als directeur wil ik een limiet op het gokbedrag, zodat het gokken niet uit de hand zal lopen."

- Acceptatiecriterium 1: bij het inzetten kan men niet een hoger bedrag dan €10,- invoeren,
- Acceptatiecriterium 2: de totale pot mag niet hoger worden dan €150,-, daarna sluit de inzet.

Definition of done

Naast specifieke acceptatiecriteria, kun je ook vastleggen waar *alle* user stories aan moeten voldoen, en waar dus het hele project ook aan moet voldoen. Dit zijn geen specifieke instructies, maar juist algemene kwaliteitseisen. Denk aan punten zoals:

- De code is getest
- De werking is gereviewd door een teamgenoot
- Voldoet aan de conventies van het bedrijf
- Enzovoort...

Deze lijst noemen we de "Definition of done", vaak afgekort tot DoD. In deze definitie ligt vast waar iedere user story aan moet voldoen. Het hele team moet het eens zijn over deze definitie.

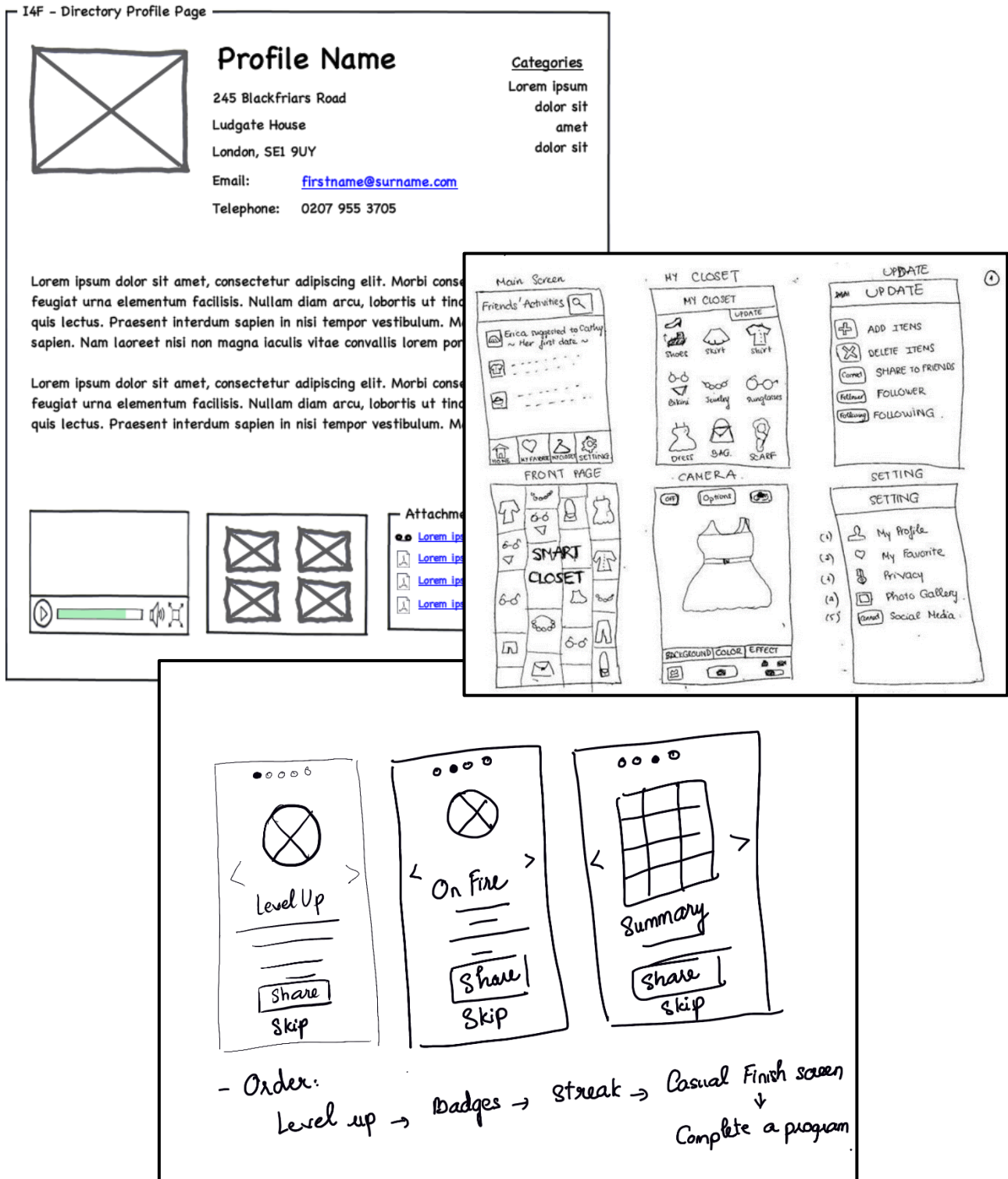
De kolom 'done'

We mogen een taak en/of user story dus pas verplaatsen naar de kolom 'done', wanneer deze voldoet aan de acceptatiecriteria én aan de 'definition of done'.

6. Wireframes

? Bron 5.1. Theorie wireframes uit ONT-I

Een wireframe is een tekening van de lay-out van een website of applicatie. Lay-out betekent: wat staat waar. Denk aan de positie van het menu, waar afbeeldingen en tekst komen te staan, positie van knoppen en invul-velden, enzovoort.



Een wireframe gaat dus *niet* over dingen als kleurgebruik, lettertypes of andere "styling". Je ziet echt alleen de lay-out terug, dus wat staat waar. Ieder scherm of iedere pagina wordt één wireframe.

Regels voor een wireframe

Er zijn een paar regels bij het maken van een wireframe:

- Werk het liefst op papier, echt tekenen dus. Digitale tools zitten het effect van snel schetsen in de weg. Je hoeft niet netjes te tekenen, het gaat erom dat de lay-out zichtbaar is.
- De belangrijkste knoppen, links en invulvelden moeten zichtbaar zijn.
- Schrijf alleen de belangrijkste woorden. Grote stukken tekst kun je weergeven met lijntjes of "lorem ipsum" onzintekst.
- Een afbeelding wordt vaak getekend als een vakje met een kruis erin.
- Geef je pagina's een naam ("home", "contactpagina"). Schrijf extra uitleg als dat nodig is.
- Maak duidelijk hoe de flow tussen je schermen is.

Een applicatie bestaat eigenlijk altijd uit meerdere schermen of pagina's. Je tekent dus ook niet één wireframe, maar er komt voor ieder scherm een tekening. Je kunt dan ook mooi aangeven hoe de "flow" in je applicatie gaat, ofwel: hoe navigeert een gebruiker van de ene pagina naar de andere.

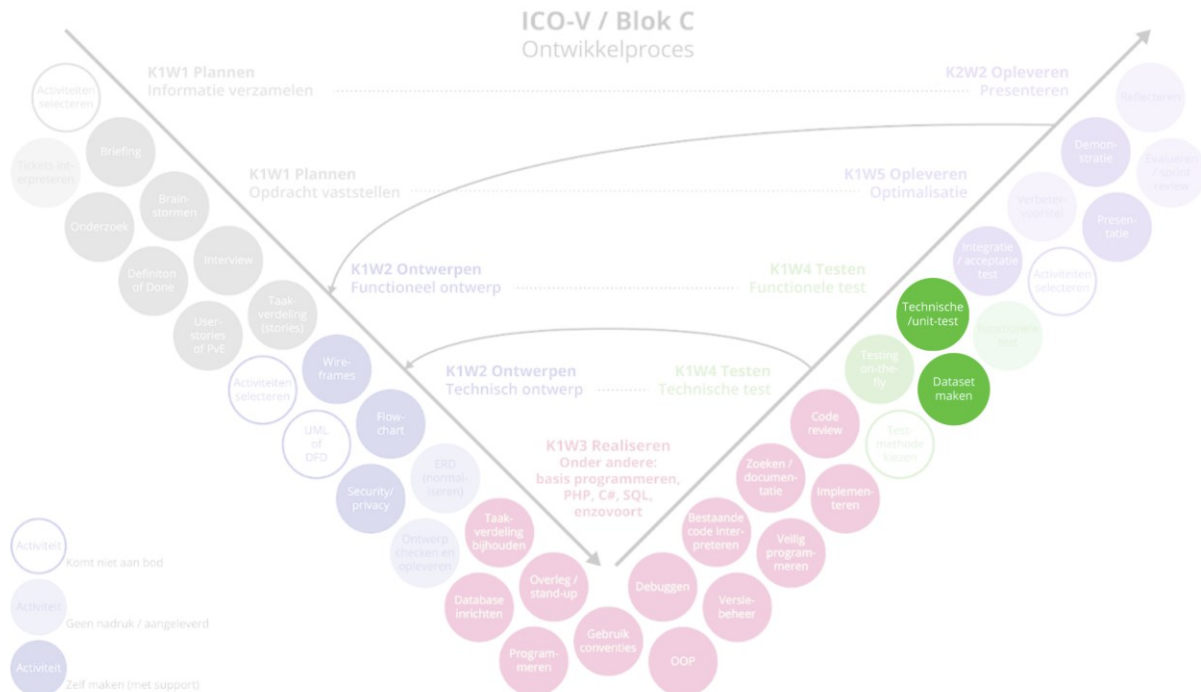
In het voorbeeld hieronder zie je dat met pijlen is aangegeven wat er gebeurt wanneer je op een link of knop drukt. De pijl wijst naar de pagina waar je dan naartoe gaat:



Je kunt dus concluderen dat het handig is om meerdere schermen op een pagina te tekenen. Een bijkomend voordeel is dat je dan wat kleiner moet tekenen, en dus niet te veel details kwijt kunt in je tekening.

7. Technisch testen

Deze module gaat over de testfase uit het V-model. Testen volgt na het programmeren. Dat kán helemaal aan het einde van een project zijn, maar natuurlijk ook aan het einde van een sprint. Daarom zie je in het V-model ook pijlen die terugverwijzen, dit geeft eigenlijk het 'iteratief' werken binnen agile / scrum aan.



Je kent al de acceptatietest, zoals geleerd in blok B. Die test is erg algemeen en kijkt vooral of alle eisen / user stories zijn gerealiseerd. Een technische test gaat veel meer in detail, je kijkt hiermee of je regels code exact doen wat ze moeten doen.



Bron 7.1. Technisch testen kennisclip

Het principe van een technische test wordt in deze kennisclip uitgelegd: <https://youtu.be/vhVhcAcobgc>.

Hieronder volgt een samenvatting van de clip:

1. Units

Een technische test kijkt naar 'units' van code, dat zijn kleine stukjes code die bij elkaar horen. Bijvoorbeeld een functie / methode of gewoon een aantal regels die samen één ding doen. Een unit heeft altijd input en verwachte output.

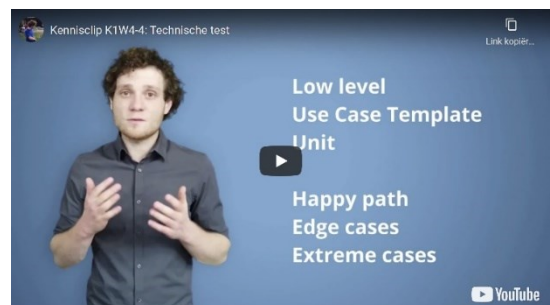
2. Use Case Template

Zie verderop in dit boekje voor de standaard "Use Case Template". Hierop vul je gegevens in over de unit die je gaat testen. Een technische test voor het hele sprint bestaat dus uit een (flink) aantal ingevulde templates. In leerjaar 3 ga je overigens leren om de Use Case Templates al bij het ontwerpen in te vullen. Je maakt dan eigenlijk de test ('waaraan moet je code voldoen') voordat je begint met programmeren!

3. Dataset

Een technische test is: input geven aan een unit van code, en kijken of er de verwachte output uitkomt. Je moet dus vastleggen welke input je wil testen, en wat je daarbij zou verwachten als output. De tester kan dan kijken of dat klopt. We testen over het algemeen drie soorten input:

- Happy path: input die juist is, zoals de code verwacht.
- Edge case: input die net niet juist is (bijvoorbeeld: negatief getal bij leeftijd).
- Extreme cases: input die nergens op slaat (bijvoorbeeld: tekst invoeren bij leeftijd).



? **Bron 7.2. Use Case Template**

Use Case / Unit <i>Naam van de use case</i>		
Pre-conditie <i>Voorwaarden voor de use case uitgevoerd kan worden</i>		
Beschrijving <i>De flow van de code (happy path)</i>	1.	
Uitzonderingen <i>Wat kan er misgaan?</i>		
Post-conditie <i>Wat is de verwachte uitkomst, wat is er gebeurd?</i>		
Test 1	Dataset	Resultaat
<i>Stappen;</i>	<i>Beschrijf dataset voor happy path;</i>	✓ ✕
<i>Verwachte uitkomst;</i>		
Test 2	Dataset	Resultaat
<i>Stappen;</i>	<i>Beschrijf dataset voor edge cases (laag);</i>	✓ ✕
<i>Verwachte uitkomst;</i>		
Test 3	Dataset	Resultaat
<i>Stappen;</i>	<i>Beschrijf dataset voor edge cases (hoog);</i>	✓ ✕
<i>Verwachte uitkomst;</i>		
Test 4	Dataset	Resultaat
<i>Stappen;</i>	<i>Beschrijf dataset voor extreme cases;</i>	✓ ✕
<i>Verwachte uitkomst;</i>		
Test 5	Dataset	Resultaat
<i>Stappen;</i>	<i>Beschrijf dataset voor ...;</i>	✓ ✕
<i>Verwachte uitkomst;</i>		

